University of Ottawa

CSI 2110 - Data Structures and Algorithms

Laboratory 5

MiniMap Route Planner: Graph ADT, Graph Traversals, Dijkstra's Algorithm, and Bellman-Ford Algorithm

Professor Mohamed Ali Ibrahim, PhD, PEng

Term: Spring/Summer 2026

Information	Details
Lab Statement Available	Tuesday, July 14, 2026
Lab Period	Tuesday, July 14 to Friday, July 24, 2026
Submission Deadline	Friday, July 24, 2026, at 11:59 PM
Submission Platform	Brightspace
Programming Language	Java
Project Type	Group programming laboratory
Required Visualization Tool	Graphviz

Important: All graph algorithms must be implemented by the group. Graphviz is permitted only for visualization.

1. Laboratory Overview

The purpose of this laboratory is to design and implement a graph-based route-planning application called MiniMap Route Planner.

The application simulates the core graph-processing functionality of a simplified navigation system. Locations such as buildings, intersections, stations, hospitals, museums, and airports are represented as vertices. Roads connecting those locations are represented as weighted edges.

The laboratory focuses on the implementation, testing, analysis, validation, and visualization of:

- the Graph Abstract Data Type;
- directed and undirected graphs;
- weighted graphs;
- adjacency-list and adjacency-matrix representations;
- Breadth-First Search and Depth-First Search;
- reachability and cycle detection;
- minimum-edge paths;
- Dijkstra's shortest-path algorithm;
- Bellman-Ford shortest-path algorithm;
- edge relaxation and priority queues;
- predecessor maps and path reconstruction;
- negative-weight edges and negative-weight cycle detection;
- Graphviz visualization;
- algorithm correctness;
- time and space complexity.

This is a programming laboratory. Each group must design, write, test, document, and demonstrate original Java code.

Installing Graphviz, generating a graph image, or calling an external shortest-path library does not satisfy the laboratory requirements.

The Graph ADT, graph representations, BFS, DFS, Dijkstra, Bellman-Ford, path reconstruction, result validation, and required tests must be implemented by the group.

2. Topics Covered in Lab 5

Chapter Area	Topics Included
Graph ADT	Vertices, edges, adjacency, incident edges, directed graphs, undirected graphs, weighted graphs, and graph operations.
Graph Representations	Adjacency lists, adjacency matrices, sparse graphs, dense graphs, and memory usage.
Graph Traversals	BFS, DFS, reachability, traversal order, predecessor trees, connected components, and cycle detection.
Shortest Paths	Path cost, relaxation, distance estimates, predecessor relationships, and shortest-path reconstruction.
Dijkstra's Algorithm	Non-negative edge weights, priority queues, finalized vertices, and stale queue entries.
Bellman-Ford Algorithm	Repeated relaxation, negative-weight edges, early termination, and negative-cycle detection.
Visualization	Graphviz DOT files, highlighted paths, visited vertices, distance labels, and negative cycles.
Validation	Path-edge validation, path-cost validation, direction validation, and algorithm cross-validation.
Complexity Analysis	Time and space complexity using adjacency lists and adjacency matrices.

3. Learning Objectives

- Define and implement a reusable Graph ADT.
- Represent a graph using an adjacency list.
- Represent the same graph using an adjacency matrix.
- Compare graph representations.
- Model a transportation network as a weighted graph.
- Load vertices and edges from external CSV files.
- Perform Breadth-First Search and reconstruct a minimum-edge path.
- Perform Depth-First Search and determine reachability.
- Detect cycles.
- Implement Dijkstra's algorithm using a priority queue.
- Implement Bellman-Ford using repeated relaxation.
- Reconstruct shortest paths from predecessor information.
- Explain why Dijkstra cannot safely process negative-weight edges.
- Detect reachable negative-weight cycles.
- Visualize algorithm results using Graphviz.
- Validate the structure and cost of a returned path.
- Compare Dijkstra and Bellman-Ford on non-negative graphs.
- Analyze the time and space complexity of graph algorithms.

4. Problem Description

A city is developing a simplified navigation application called MiniMap Route Planner.

The application represents locations such as the University of Ottawa, Rideau Centre, Parliament, Train Station, South Keys, General Hospital, Ottawa Airport, Downtown, Kanata, Orleans, Gatineau, and a museum.

Each location is represented by a vertex. Each road is represented by a directed or undirected weighted edge. A road may contain distance in kilometres, estimated travel time in minutes, and adjusted transportation cost.

The application must answer questions such as:

- Which locations are directly connected?
- Which locations are reachable from a selected source?
- What are the BFS and DFS traversal orders?
- Does the network contain a cycle?
- What path contains the smallest number of roads?
- What route has the smallest total distance?
- What route has the smallest total travel time?
- What happens when the graph contains a negative-weight edge?
- Does the graph contain a reachable negative-weight cycle?
- Do Dijkstra and Bellman-Ford produce the same result on a non-negative graph?

The project is inspired by navigation applications, but it is not intended to reproduce the complete functionality or interface of Google Maps. The main purpose is to implement and analyze graph data structures and algorithms. A complex graphical user interface is not required.

5. Required Data Models

5.1 Location

A class equivalent to the following must be implemented:

```
public class Location {
    private final String id;
    private final String name;

    public Location(String id, String name) {
```

```
        this.id = id;
        this.name = name;
    }

    public String getId() { return id; }
    public String getName() { return name; }

    // Implement equals(), hashCode(), and toString().
}
```

Each location must have a unique identifier.

5.2 Weighted Edge

```
public class WeightedEdge<V> {
    private final V source;
    private final V destination;
    private final double weight;

    public WeightedEdge(V source, V destination, double weight) {
        this.source = source;
        this.destination = destination;
        this.weight = weight;
    }

    public V getSource() { return source; }
    public V getDestination() { return destination; }
    public double getWeight() { return weight; }
}
```

5.3 Road

```
public class Road {
    private final Location source;
    private final Location destination;
    private final double distance;
    private final double travelTime;
    private final double adjustedCost;
    private final boolean directed;

    // Constructor, getters, and toString().
}
```

A generic WeightedEdge<V> design or a specialized Road design is acceptable, provided that the design is clear and documented.

6. Suggested Project Structure

```
CSI2110_Lab5/
|
+-- src/
|   +-- graph/
|   |   +-- Graph.java
|   |   +-- WeightedEdge.java
|   |   +-- AdjacencyListGraph.java
|   |   +-- AdjacencyMatrixGraph.java
|   +-- algorithms/
|   |   +-- BreadthFirstSearch.java
|   |   +-- DepthFirstSearch.java
|   |   +-- Dijkstra.java
|   |   +-- BellmanFord.java
|   |   +-- ShortestPathResult.java
|   +-- minimap/
|   |   +-- Location.java
|   |   +-- Road.java
|   |   +-- MapLoader.java
|   |   +-- PathValidator.java
```

```
|   |   +-- GraphvizExporter.java
|   +-- application/
|       +-- Main.java
+-- data/
|   +-- locations.csv
|   +-- roads_positive.csv
|   +-- roads_negative.csv
|   +-- roads_negative_cycle.csv
+-- output/
|   +-- graph.dot / graph.svg
|   +-- bfs_result.dot / bfs_result.svg
|   +-- dfs_result.dot / dfs_result.svg
|   +-- dijkstra_result.dot / dijkstra_result.svg
|   +-- bellman_ford_result.dot / bellman_ford_result.svg
|   +-- result.json
|   +-- trace.csv
+-- test/
|   +-- GraphTests.java
|   +-- TraversalTests.java
|   +-- ShortestPathTests.java
+-- README.md
+-- Lab5_Report.pdf
```

Equivalent structures are acceptable when they remain clear and logically organized.

7. Part A - Graph ADT

Implement a reusable Graph ADT. A possible interface is shown below:

```
public interface Graph<V> {
    boolean addVertex(V vertex);
    boolean removeVertex(V vertex);
    boolean addEdge(V source, V destination, double weight);
    boolean removeEdge(V source, V destination);
    boolean containsVertex(V vertex);
    boolean containsEdge(V source, V destination);
    List<V> getVertices();
    List<V> getNeighbors(V vertex);
    List<WeightedEdge<V>> getOutgoingEdges(V vertex);
    int vertexCount();
    int edgeCount();
    boolean isDirected();
}
```

The exact method names may differ, but all required operations must be supported.

- Ensure that vertices are uniquely identifiable.
- Handle duplicate vertices correctly.
- Handle duplicate edges according to a documented policy.
- Support weighted edges and directed graphs.
- Explain how undirected edges are represented.
- Prevent inappropriate direct modification of internal structures.
- Handle invalid vertices safely.
- Provide meaningful error messages or documented return values.
- Provide a readable graph representation.

8. Part B - Graph Representations

8.1 Adjacency-List Representation

```
public class AdjacencyListGraph<V> implements Graph<V> {
    private Map<V, List<WeightedEdge<V>>> adjacencyList;
    // Required implementation
}
```

- Add and remove vertices.
- Add and remove edges.
- Retrieve outgoing edges and neighboring vertices.
- Test whether an edge exists.
- Retrieve vertex and edge counts.
- Support directed weighted edges.
- Produce readable output.

8.2 Adjacency-Matrix Representation

```
public class AdjacencyMatrixGraph<V> implements Graph<V> {
    private List<V> vertices;
    private double[][] matrix;
    // Required implementation
}
```

The implementation must define a clear value for the absence of an edge, such as Double.POSITIVE_INFINITY. The diagonal should normally contain 0.0.

8.3 Required Representation Comparison

The report must compare adjacency lists and adjacency matrices according to:

- memory usage;
- edge-existence testing;
- edge insertion and removal;
- neighbor iteration;
- suitability for sparse graphs;
- suitability for dense graphs;
- suitability for road networks.

The main MiniMap application should use the adjacency-list representation. The adjacency-matrix implementation remains mandatory for implementation, demonstration, and comparison.

9. Part C - Loading the MiniMap Network

The graph must be loaded from external CSV files. Hard-coding the complete required graph directly inside Main.java is not acceptable.

9.1 Locations File

```
id,name
UO,University of Ottawa
RC,Rideau Centre
PA,Parliament
TS,Train Station
SK,South Keys
AP,Ottawa Airport
HO,General Hospital
```

9.2 Roads File

```
source,destination,distance,travelTime,adjustedCost,directed
UO,RC,1.8,6,1.8,true
RC,PA,1.2,5,1.2,true
UO,TS,4.5,9,4.5,true
TS,SK,5.7,11,5.7,true
SK,AP,6.2,9,6.2,true
HO,UO,3.4,8,3.4,false
```

9.3 Loader Requirements

- Read all location records and create the corresponding vertices.
- Read all road records and create the corresponding weighted edges.
- Validate referenced location identifiers.
- Handle invalid numeric values, empty files, and missing columns.
- Handle duplicate locations and duplicate roads.
- Distinguish directed and undirected roads.
- Display meaningful error messages.

For an undirected road, the implementation may insert one directed edge in each direction.

10. Required Datasets

10.1 Dataset 1 - Standard Transportation Network

- at least 15 vertices;
- at least 25 edges;
- only non-negative weights;
- multiple routes between several pairs of locations;
- at least one directed one-way road;
- at least one undirected road;
- at least one cycle;
- at least one location unreachable from a selected source;
- at least two routes with different numbers of edges and different total costs.

This dataset is used for adjacency-list and adjacency-matrix testing, BFS, DFS, Dijkstra, Bellman-Ford cross-validation, and Graphviz visualization.

10.2 Dataset 2 - Negative Edge Without a Negative Cycle

- at least 6 vertices;
- at least 10 directed edges;
- at least one negative-weight edge;
- no reachable negative-weight cycle.

The negative value represents an adjusted transportation cost, not a physical distance. For example: adjusted cost = normal transportation cost - transportation credit.

10.3 Dataset 3 - Reachable Negative-Weight Cycle

- at least 4 vertices;
- at least one reachable negative-weight cycle;
- a clearly identified source vertex;
- at least one destination affected by the cycle.

Bellman-Ford must detect and report the cycle.

11. Complete Use Case - Input, Processing, and Graph Output

This example demonstrates how a dataset is loaded, processed, validated, and visualized. The submitted project must not be limited to this small example.

11.1 Scenario

A user wants to travel from the University of Ottawa to Ottawa Airport. The application must:

1. load the locations;
2. load the roads;
3. construct a weighted directed graph;
4. display the adjacency list and adjacency matrix;
5. use BFS to find a path containing the minimum number of roads;

6. use Dijkstra to find a route having the minimum total distance;
7. validate the Dijkstra path;
8. export the result in Graphviz DOT format;
9. generate an SVG or PNG visualization.

11.2 Example Input File - Locations

```
id,name
UO,University of Ottawa
RC,Rideau Centre
PA,Parliament
TS,Train Station
SK,South Keys
AP,Ottawa Airport
HO,General Hospital
```

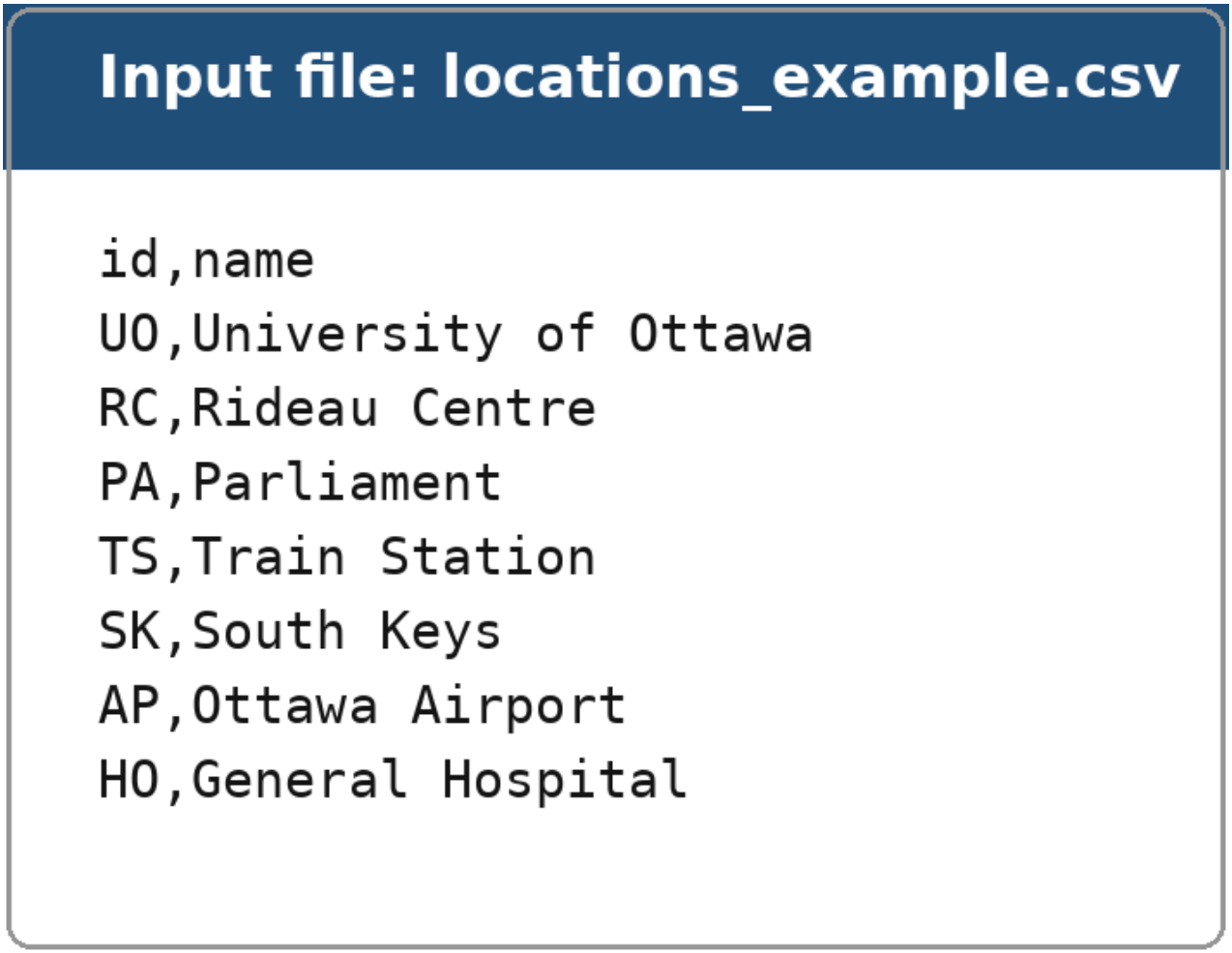


Figure 1 - Example input image for locations_example.csv.

11.3 Example Input File - Roads

```
source,destination,distance,travelTime,adjustedCost,directed
UO,RC,1.8,6,1.8,true
RC,PA,1.2,5,1.2,true
PA,AP,20.0,30,20.0,true
UO,TS,4.5,9,4.5,true
TS,SK,5.7,11,5.7,true
SK,AP,6.2,9,6.2,true
UO,AP,25.0,35,25.0,true
RC,TS,6.0,12,6.0,true
HO,UO,3.4,8,3.4,false
```

Input file: roads_example.csv

```
source,destination,distance,travelTime,adjustedCost,directed
UO,RC,1.8,6,1.8,true
RC,PA,1.2,5,1.2,true
PA,AP,20.0,30,20.0,true
UO,TS,4.5,9,4.5,true
TS,SK,5.7,11,5.7,true
SK,AP,6.2,9,6.2,true
UO,AP,25.0,35,25.0,true
RC,TS,6.0,12,6.0,true
HO,UO,3.4,8,3.4,false
```

Figure 2 - Example input image for roads_example.csv.

The last road is undirected. The internal graph must therefore contain both HO -> UO and UO -> HO.

11.4 Graph Constructed from the Input

```
UO --1.8--> RC --1.2--> PA --20.0--> AP
 |            |
 |            +--6.0--> TS
 |
 +--4.5--> TS --5.7--> SK --6.2--> AP
 |
 +--25.0-------------------------> AP
 |
 +--3.4<-------------------------> HO
```

11.5 Expected Adjacency List

```
ADJACENCY-LIST REPRESENTATION

University of Ottawa:
    -> Rideau Centre         distance = 1.8
    -> Train Station         distance = 4.5
    -> Ottawa Airport        distance = 25.0
    -> General Hospital      distance = 3.4

Rideau Centre:
    -> Parliament            distance = 1.2
    -> Train Station         distance = 6.0

Parliament:
    -> Ottawa Airport        distance = 20.0

Train Station:
    -> South Keys            distance = 5.7

South Keys:
    -> Ottawa Airport        distance = 6.2

Ottawa Airport:
    No outgoing roads
```

```
General Hospital:
    -> University of Ottawa  distance = 3.4
```

11.6 Expected Adjacency Matrix

Vertex order: UO, RC, PA, TS, SK, AP, HO. The symbol INF indicates that no direct edge exists.

From/To	UO	RC	PA	TS	SK	AP	HO
UO	0	1.8	INF	4.5	INF	25.0	3.4
RC	INF	0	1.2	6.0	INF	INF	INF
PA	INF	INF	0	INF	INF	20.0	INF
TS	INF	INF	INF	0	5.7	INF	INF
SK	INF	INF	INF	INF	0	6.2	INF
AP	INF	INF	INF	INF	INF	0	INF
HO	3.4	INF	INF	INF	INF	INF	0

11.7 Deterministic Neighbor Order

The group must define and document the order used to process neighbors. For this example, assume that neighbors are processed alphabetically by identifier. The neighbors of UO are processed in this order: AP, HO, RC, TS. A different deterministic order is acceptable when it is documented and applied consistently.

11.8 Expected BFS Result

```
BREADTH-FIRST SEARCH

Source: University of Ottawa
Destination: Ottawa Airport

Traversal order:
University of Ottawa
Ottawa Airport
General Hospital
Rideau Centre
Train Station
Parliament
South Keys

Minimum-edge path:
University of Ottawa -> Ottawa Airport

Number of edges: 1
Total distance of returned path: 25.0 km
Destination reachable: Yes
```

BFS minimizes the number of edges. It does not minimize the total weight.

11.9 Dijkstra Route Comparison

Route	Path	Total Distance
Direct route	UO -> AP	25.0
Parliament route	UO -> RC -> PA -> AP	23.0
South Keys route	UO -> TS -> SK -> AP	16.4
Rideau-Train route	UO -> RC -> TS -> SK -> AP	19.7

Dijkstra must therefore return UO -> TS -> SK -> AP with a total distance of 16.4 km.

11.10 Expected Dijkstra Result

```
DIJKSTRA'S SHORTEST-PATH ALGORITHM

Source: University of Ottawa
Destination: Ottawa Airport
Optimization metric: Distance

Shortest path:
University of Ottawa
-> Train Station
-> South Keys
-> Ottawa Airport

Number of edges: 3
Total distance: 16.4 km
Destination reachable: Yes
```

11.11 Expected Final Distance Table

Vertex	Final Distance	Predecessor
UO	0.0	None
RC	1.8	UO
PA	3.0	RC
TS	4.5	UO
SK	10.2	TS
AP	16.4	SK
HO	3.4	UO

Path reconstruction begins with AP <- SK <- TS <- UO. After reversing the sequence, the path is UO -> TS -> SK -> AP.

11.12 Expected Dijkstra Trace

```
STEP 1 - Finalize UO at 0.0
Relax UO -> RC: Infinity becomes 1.8; predecessor[RC] = UO
Relax UO -> TS: Infinity becomes 4.5; predecessor[TS] = UO
Relax UO -> AP: Infinity becomes 25.0; predecessor[AP] = UO
Relax UO -> HO: Infinity becomes 3.4; predecessor[HO] = UO

STEP 2 - Finalize RC at 1.8
Relax RC -> PA: Infinity becomes 3.0; predecessor[PA] = RC
Relax RC -> TS: candidate 7.8; no update

STEP 3 - Finalize PA at 3.0
Relax PA -> AP: 25.0 becomes 23.0; predecessor[AP] = PA

STEP 4 - Finalize HO at 3.4
Relax HO -> UO: candidate 6.8; no update

STEP 5 - Finalize TS at 4.5
Relax TS -> SK: Infinity becomes 10.2; predecessor[SK] = TS

STEP 6 - Finalize SK at 10.2
Relax SK -> AP: 23.0 becomes 16.4; predecessor[AP] = SK

STEP 7 - Finalize AP at 16.4
Destination finalized; algorithm terminates.
```

11.13 BFS and Dijkstra Comparison

Algorithm	Returned Path	Number of Edges	Total Distance
BFS	UO -> AP	1	25.0 km
Dijkstra	UO -> TS -> SK -> AP	3	16.4 km

This demonstrates that BFS minimizes the number of edges, while Dijkstra minimizes the total weight. The minimum-edge path is not necessarily the minimum-distance path.

11.14 Expected Graphviz DOT Output

The application must generate a DOT file that contains all vertices, all roads, all edge weights, the final distances, and the highlighted shortest path.

```
digraph MiniMapDijkstra {
    rankdir=LR;
    splines=true;
    overlap=false;

    graph [
        label="MiniMap Route Planner – Dijkstra Result",
        labelloc="t",
        fontsize=20,
        fontname="Arial",
        pad=0.3,
        nodesep=0.7,
        ranksep=1.0
    ];

    node [
        shape=ellipse,
        style=filled,
        fillcolor="white",
        fontname="Arial",
        fontsize=11
    ];

    edge [
        fontname="Arial",
        fontsize=10,
        arrowsize=0.8
    ];

    UO [label="University of Ottawa\nDistance: 0.0",
        fillcolor="palegreen", penwidth=3];

    RC [label="Rideau Centre\nDistance: 1.8"];

    PA [label="Parliament\nDistance: 3.0"];

    TS [label="Train Station\nDistance: 4.5",
        fillcolor="lightgoldenrod1"];

    SK [label="South Keys\nDistance: 10.2",
        fillcolor="lightgoldenrod1"];

    AP [label="Ottawa Airport\nDistance: 16.4",
        shape=doublecircle, fillcolor="lightblue", penwidth=3];

    HO [label="General Hospital\nDistance: 3.4"];

    UO -> RC [label="1.8", color="gray50"];
    RC -> PA [label="1.2", color="gray50"];
    PA -> AP [label="20.0", color="gray50"];

    UO -> TS [label="4.5", color="red", penwidth=4];
    TS -> SK [label="5.7", color="red", penwidth=4];
    SK -> AP [label="6.2", color="red", penwidth=4];

    UO -> AP [label="25.0", color="gray50"];
    RC -> TS [label="6.0", color="gray50"];

    UO -> HO [label="3.4", color="gray50", dir=both];
}
```

Figure 3 - Example Graphviz DOT output generated by the application.

11.15 Expected Visual Graph Output

The generated graph must identify UO as the source, AP as the destination, display all road weights and final distances, and highlight the shortest path UO -> TS -> SK -> AP.

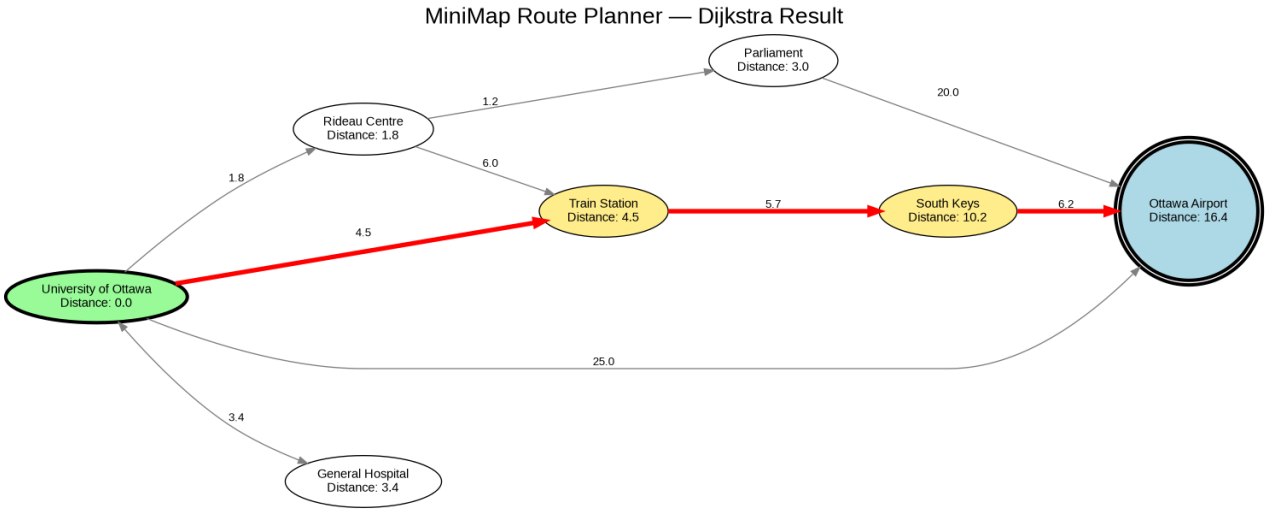


Figure 4 - Expected visual output for Dijkstra's shortest path.

11.16 Graphviz Conversion Commands

```
dot -Tsvg dijkstra_example.dot -o dijkstra_example.svg

dot -Tpng dijkstra_example.dot -o dijkstra_example.png
```

SVG is recommended because it remains clear when enlarged.

11.17 Expected JSON Result


```
{
  "algorithm": "DIJKSTRA",
  "metric": "DISTANCE",
  "source": "UO",
  "destination": "AP",
  "reachable": true,
  "path": ["UO", "TS", "SK", "AP"],
  "numberOfEdges": 3,
  "totalCost": 16.4,
  "distances": {
    "UO": 0.0,
    "RC": 1.8,
    "PA": 3.0,
    "TS": 4.5,
    "SK": 10.2,
    "AP": 16.4,
    "HO": 3.4
  },
  "predecessors": {
    "RC": "UO",
    "PA": "RC",
    "TS": "UO",
    "SK": "TS",
    "AP": "SK",
    "HO": "UO"
  },
  "validation": {
    "validSource": true,
    "validDestination": true,
    "validEdges": true,
    "validDirections": true,
    "validCost": true,
    "overallResult": "VALID"
```

```
  }
}
```

11.18 Expected Validation Result

```
PATH VALIDATION REPORT

Requested source: UO
Requested destination: AP
Path: UO -> TS -> SK -> AP

PASS: UO exists in the graph
PASS: AP exists in the graph
PASS: Path starts at UO
PASS: Path ends at AP
PASS: Edge UO -> TS exists
PASS: Edge TS -> SK exists
PASS: Edge SK -> AP exists
PASS: All edge directions are valid

Calculated cost: 4.5 + 5.7 + 6.2 = 16.4
Reported cost: 16.4
PASS: Calculated cost equals reported cost
PASS: Destination distance equals path cost

Overall result: VALID
```

11.19 Acceptance Criteria for the Use Case

- all seven locations are loaded;
- all roads are loaded;
- the undirected road creates both directions;
- the adjacency list and matrix are correct;
- BFS returns UO -> AP and reports one edge;
- Dijkstra returns UO -> TS -> SK -> AP and reports 16.4;
- Bellman-Ford also reports 16.4;
- the path validator reports VALID;
- the DOT and SVG or PNG files are generated;
- the highlighted path matches the Dijkstra result;
- the JSON result contains the correct path and cost.

12. Part D - Breadth-First Search

Implement BFS without using an external graph-algorithm library.

```
public List<V> breadthFirstTraversal(Graph<V> graph, V start)

public List<V> breadthFirstPath(
    Graph<V> graph,
    V source,
    V destination
)
```

The BFS implementation must use a queue, a visited set, a predecessor map, a deterministic neighbor order, and path reconstruction.

- BFS traversal order;
- BFS levels;
- all reachable and unreachable vertices;
- a minimum-edge path;

- source equal to destination;
- unreachable destination;
- invalid source;
- disconnected graph;
- graph containing cycles.

BFS must not be presented as a weighted shortest-path algorithm.

13. Part E - Depth-First Search

Implement DFS recursively or iteratively.

```
public List<V> depthFirstTraversal(Graph<V> graph, V start)

public boolean isReachable(
    Graph<V> graph,
    V source,
    V destination
)

public boolean hasCycle(Graph<V> graph)
```

- traversal order;
- reachable and unreachable vertices;
- reachability testing;
- cycle detection;
- connected and disconnected graphs;
- directed and undirected cycles;
- acyclic graph;
- isolated vertex;
- self-loop;
- empty graph.

The report must explain why a DFS path is not guaranteed to minimize the number of edges, total distance, or total travel time. If cycle detection differs between directed and undirected graphs, the distinction must be explained.

14. Part F - Dijkstra's Algorithm

Implement Dijkstra's shortest-path algorithm for graphs containing non-negative edge weights.

```
public ShortestPathResult<V> dijkstra(
    Graph<V> graph,
    V source,
    V destination
)
```

```
public class ShortestPathResult<V> {
    private final List<V> path;
    private final double totalCost;
    private final Map<V, Double> distances;
    private final Map<V, V> predecessors;
    private final boolean reachable;
    // Constructor and getters
}
```

- distance and predecessor initialization;
- a priority queue;
- edge relaxation;
- stale priority-queue entry handling or an equivalent strategy;
- destination processing;

- unreachable-destination handling;
- path reconstruction;
- negative-edge protection;
- a readable algorithm trace.

Dijkstra must not silently process a graph containing a negative-weight edge. The program must reject the execution with a clear message and recommend Bellman-Ford.

14.1 Optimization Metrics

The application must support at least two of the following:

- shortest distance;
- shortest travel time;
- smallest non-negative adjusted cost.

14.2 Trace Requirements

- current vertex and current distance;
- source and destination of the processed edge;
- old distance;
- candidate distance;
- new distance;
- whether an update occurred;
- predecessor update.

The trace must be printed and may also be stored in trace.csv.

15. Part G - Bellman-Ford Algorithm

Implement Bellman-Ford without using an external graph library.

```
public ShortestPathResult<V> bellmanFord(
    Graph<V> graph,
    V source,
    V destination
)
```

1. initialize all distances and assign zero to the source;
2. perform up to |V| - 1 relaxation passes;
3. relax every edge during each pass;
4. maintain predecessor information;
5. reconstruct valid shortest paths;
6. terminate early when an entire pass produces no update;
7. perform an additional pass for negative-cycle detection;
8. report reachable negative-weight cycles;
9. distinguish reachable and unreachable negative cycles;
10. avoid reporting a finite shortest path when one does not exist.

```
BELLMAN-FORD ALGORITHM

Source: A
Destination: F

Pass 1: 7 updates
Pass 2: 4 updates
Pass 3: 1 update
Pass 4: 0 updates

Early termination applied.
Shortest path: A -> C -> D -> F
Total adjusted cost: 5.0
```

```
Reachable negative-weight cycle: No
```

```
BELLMAN-FORD ALGORITHM

Source: A
Negative-weight cycle detected.
Cycle: B -> C -> D -> B

A finite shortest-path result cannot be reported
for destinations affected by this cycle.
```

The report must explain why a reachable negative-weight cycle prevents a finite shortest-path value.

16. Part H - Graph Visualization

The application must export graph data using Graphviz DOT format. Graphviz may be used only for visualization and must not be used to compute BFS, DFS, reachability, cycle detection, Dijkstra, Bellman-Ford, shortest-path costs, or shortest-path trees.

16.1 Required DOT Files

```
graph.dot
bfs_result.dot
dfs_result.dot
dijkstra_result.dot
bellman_ford_result.dot
```

The corresponding SVG or PNG files must also be submitted.

16.2 Required Visual Distinctions

Element	Required Treatment
Source vertex	Distinct fill or border
Destination vertex	Distinct fill or double border
Visited vertices	Distinct fill
Unvisited vertices	Neutral appearance
Ordinary edges	Standard or secondary line
Final path edges	Thick highlighted line
Negative edges	Distinct line or label
Negative cycle	Clearly highlighted
Edge weights	Visible labels
Final distances	Visible inside or near vertices

The group may select its own colors, provided that the graph remains readable.

17. Part I - Path Validation and Algorithm Cross-Validation

17.1 Path Validation

Implement a PathValidator class or an equivalent component. For every path returned by BFS, Dijkstra, or Bellman-Ford, the validator must check:

- the source and destination exist;

- the path starts at the requested source;
- the path ends at the requested destination;
- every consecutive pair is connected;
- every edge direction is valid;
- the number of edges is correct;
- the reported cost equals the sum of path weights;
- unreachable results contain no fabricated path.

Floating-point values must be compared using a tolerance, for example:

```
private static final double EPSILON = 1.0e-9;
```

17.2 Dijkstra and Bellman-Ford Cross-Validation

On the standard non-negative dataset, both algorithms must be executed from the same source. For every reachable destination, both algorithms must report the same shortest-path cost.

Destination	Dijkstra Cost	Bellman-Ford Cost	Match?
Rideau Centre			
Parliament			
Train Station			
Airport			

A different valid path may be accepted when several paths have the same shortest cost.

18. Required Application Menu

A console application is sufficient. The application must provide functionality equivalent to:

```
========== MINIMAP ROUTE PLANNER ==========

1. Load a graph dataset
2. Display all locations
3. Display all roads
4. Display the adjacency list
5. Display the adjacency matrix
6. Run Breadth-First Search
7. Find a minimum-edge path using BFS
8. Run Depth-First Search
9. Test reachability
10. Detect cycles
11. Find a shortest route using Dijkstra
12. Find a shortest route using Bellman-Ford
13. Detect a negative-weight cycle
14. Validate the last returned path
15. Export Graphviz files
16. Compare Dijkstra and Bellman-Ford
17. Exit

Select an option:
```

A graphical interface is optional and does not replace any required component.

19. Complexity Analysis

The report must complete the following table. Define V as the number of vertices and E as the number of edges.

Algorithm or Operation	Adjacency-List Time	Adjacency-Matrix Time	Space Complexity

Algorithm or Operation	Adjacency-List Time	Adjacency-Matrix Time	Space Complexity
Test whether an edge exists			
Add a vertex			
Add an edge			
Remove an edge			
Iterate through neighbors			
BFS			
DFS			
Dijkstra with priority queue			
Dijkstra using a matrix			
Bellman-Ford			
Path reconstruction			

19.1 Required Written Questions

1. What does a vertex represent in MiniMap?
2. What does an edge represent?
3. Why is an adjacency list generally appropriate for a road network?
4. When could an adjacency matrix be preferable?
5. Why does BFS find a minimum-edge path?
6. Why does BFS not necessarily find a minimum-weight path?
7. Why does DFS not guarantee a shortest path?
8. What is edge relaxation?
9. What is the purpose of a predecessor map?
10. What is the purpose of the priority queue in Dijkstra?
11. Why must Dijkstra reject negative-weight edges?
12. How does Bellman-Ford process negative-weight edges?
13. Why are at most |V| - 1 main relaxation passes required?
14. How does Bellman-Ford detect a reachable negative-weight cycle?
15. Why should Dijkstra and Bellman-Ford return equal costs on a non-negative graph?
16. What is the difference between minimizing the number of roads and minimizing total distance?
17. How are unreachable vertices represented?
18. What are the main complexity differences between Dijkstra and Bellman-Ford?
19. What effect does graph density have on the choice of representation?
20. Why is visual inspection insufficient as the only correctness test?

20. Testing Requirements

A few successful console executions are not sufficient. Systematic tests must be implemented.

20.1 Graph ADT Tests

- Empty graph.
- One-vertex graph.
- Multiple vertex insertion.
- Duplicate vertex.

- Valid edge insertion.
- Missing source.
- Missing destination.
- Duplicate edge.
- Directed edge.
- Undirected edge.
- Self-loop.
- Edge removal.
- Vertex removal.
- Invalid weight.
- Empty neighbor list.

20.2 BFS Tests

- Connected graph.
- Disconnected graph.
- Source equal to destination.
- Unreachable destination.
- Invalid source.
- Several routes with different edge counts.
- Graph containing a cycle.
- Isolated vertex.
- Empty graph.

20.3 DFS Tests

- Connected graph.
- Disconnected graph.
- Directed cycle.
- Undirected cycle.
- Acyclic graph.
- Self-loop.
- Isolated vertex.
- Empty graph.
- Invalid source.

20.4 Dijkstra Tests

- One possible route.
- Several competing routes.
- Equal-cost routes.
- Zero-weight edge.
- Unreachable destination.
- Source equal to destination.
- Negative-weight edge rejection.
- Minimum-edge path different from minimum-weight path.
- Stale priority-queue entry.
- Disconnected graph.

20.5 Bellman-Ford Tests

- All-positive graph.
- Zero-weight edge.
- One negative edge.
- Several negative edges.

- Negative edge without a negative cycle.
- Reachable negative cycle.
- Unreachable negative cycle.
- Unreachable destination.
- Early termination.
- Source equal to destination.

20.6 Visualization Tests

- All vertices appear.
- All edges appear.
- Edge directions are correct.
- Weights are correct.
- The highlighted path matches the returned path.
- Source and destination are distinguishable.
- Negative edges are distinguishable.
- The negative cycle is highlighted.
- Unreachable vertices are not included in the path.
- Final distances are displayed correctly.

21. Java Library and External Tool Restrictions

The following standard Java utilities may be used:

```
ArrayList, LinkedList, ArrayDeque, HashMap,
LinkedHashMap, HashSet, LinkedHashSet, PriorityQueue,
Comparator, Collections, Scanner, File,
BufferedReader, and BufferedWriter
```

Equivalent standard Java utilities are permitted.

The following are not permitted for implementing the required algorithms:

- JGraphT shortest-path algorithms;
- Apache Commons Graph;
- Guava graph algorithms;
- GraphStream shortest-path algorithms;
- Google Maps route calculation or Google Maps Directions API;
- external BFS, DFS, Dijkstra, or Bellman-Ford implementations;
- external services that return calculated routes.

Java's PriorityQueue is permitted as part of the group's own Dijkstra implementation. Graphviz is permitted only for visualization.

22. Deliverables

Each group must submit one ZIP file on Brightspace. Suggested filename: CSI2110_Lab5_GroupXX.zip.

- all Java source files;
- all input CSV files;
- all required DOT files;
- all required SVG or PNG files;
- result.json;
- trace.csv;
- all test files;
- README.md;
- the laboratory report in PDF format.

22.1 Required Source Components

```
Graph.java
WeightedEdge.java
AdjacencyListGraph.java
AdjacencyMatrixGraph.java
Location.java
Road.java
MapLoader.java
BreadthFirstSearch.java
DepthFirstSearch.java
Dijkstra.java
BellmanFord.java
ShortestPathResult.java
PathValidator.java
GraphvizExporter.java
Main.java
```

22.2 README Requirements

- Java version used;
- project structure;
- compilation instructions;
- execution instructions;
- dataset selection;
- algorithm execution;
- test execution;
- DOT generation;
- SVG or PNG conversion;
- implementation assumptions;
- known limitations, if any.

22.3 Report Requirements

- team member names and student numbers;
- description of the Graph ADT;
- description and comparison of both graph representations;
- class diagram or architecture diagram;
- explanations of BFS, DFS, Dijkstra, and Bellman-Ford;
- explanation of path reconstruction and Graphviz export;
- screenshots of important outputs and visual graphs;
- completed complexity table;
- Dijkstra-Bellman-Ford comparison table;
- answers to all written questions;
- test strategy;
- encountered problems and solutions;
- division of work among group members;
- conclusion.

23. Laboratory Demonstration Requirement

The laboratory demonstration is mandatory. Each group must demonstrate the submitted implementation to the Teaching Assistant.

1. loading a graph from CSV;
2. displaying the adjacency list and adjacency matrix;
3. running BFS and reconstructing a minimum-edge path;
4. running DFS and testing reachability;

5. detecting a cycle;
6. running Dijkstra and Bellman-Ford;
7. rejecting Dijkstra on a negative-edge dataset;
8. processing a negative edge with Bellman-Ford;
9. detecting a reachable negative-weight cycle;
10. validating a returned path;
11. generating a Graphviz visualization;
12. comparing Dijkstra and Bellman-Ford.

All group members must be present. Each member may be asked to explain a class or method, trace an algorithm manually, identify an operation complexity, modify a test case, predict an output, explain a failed test, or explain the difference between BFS, Dijkstra, and Bellman-Ford.

If a group does not complete the mandatory demonstration, the maximum mark that the group can receive is 50%. This maximum applies only to an exceptionally complete and correct submission. In general, without the required demonstration, the final grade is expected to be significantly lower than 50%.

24. Grading Rubric and Evaluation Criteria

Total: 100 points

Component	Points	Evaluation Criteria
Graph ADT design and correctness	8	Appropriate interface; correct vertex and edge operations; duplicate handling; error handling; abstraction and encapsulation.
Adjacency-list representation	7	Correct weighted representation; correct insertion, deletion, lookup, and neighbor retrieval; readable output.
Adjacency-matrix representation	6	Correct vertex-index mapping; absent-edge representation; updates; displayed matrix.
Representation comparison	3	Accurate analysis of memory, operation costs, sparse/dense graphs, and road-network suitability.
CSV loading and data validation	5	Correct parsing; directed and undirected roads; invalid-row and duplicate handling; meaningful errors.
Breadth-First Search	9	Correct queue-based traversal; visited structure; deterministic order; BFS levels; predecessor map.
Minimum-edge path reconstruction	3	Correct reconstruction; source/destination handling; unreachable case; correct edge count.
Depth-First Search	6	Correct recursive or iterative traversal; visited handling; deterministic and complete output.
Reachability and cycle detection	4	Correct reachability; appropriate directed or undirected cycle detection; edge cases.
Dijkstra's algorithm	15	Correct initialization, priority queue, relaxation, stale-entry handling, predecessor updates, path reconstruction, and unreachable handling.
Dijkstra negative-edge protection	2	Negative edges are detected and rejected with a clear explanation.
Bellman-Ford algorithm	11	Correct repeated relaxation; V - 1 passes; predecessor handling; path reconstruction; early termination.
Negative-weight cycle detection	4	Correct extra pass; reachable-cycle detection; appropriate result reporting and explanation.
Graphviz visualization	5	Correct DOT export; all vertices and edges; labels and directions; highlighted source, destination, final path, and negative cycle.
Path validation	3	Correct endpoint, edge, direction, edge-count, and total-cost checks.
Dijkstra/Bellman-Ford cross-validation	2	Correct comparison on non-negative data; floating-point tolerance; meaningful result table.
Testing and edge cases	3	Systematic tests across all required categories; clear expected and actual results.
Complexity analysis and written answers	2	Correct Big-O analysis and concise, technically accurate explanations.

Component	Points	Evaluation Criteria
Code quality, documentation, README, and report	2	Clear naming, comments, modular organization, compilation instructions, and readable report.
Total	100	

24.1 General Performance Levels

Excellent: The component is complete and correct; all required cases are handled; the design is modular and clearly explained; tests demonstrate normal and exceptional behavior; outputs and visualizations are accurate; the group can explain the implementation.

Satisfactory: The principal functionality works; some secondary cases or explanations are incomplete; minor design, output, or testing issues remain; the group demonstrates reasonable understanding.

Limited: Only part of the required functionality works; important cases are missing; testing is weak; significant design or correctness problems remain; explanations are incomplete.

Insufficient: The component is missing or does not execute; results are hard-coded or produced by a prohibited library; the group cannot explain the submitted code; required files, tests, or outputs are absent.

24.2 Correctness Expectations

A solution is not considered fully correct merely because it produces the expected result for the supplied example. The implementation must also work on different vertex names, different graph sizes, different insertion orders, disconnected graphs, alternative valid shortest paths, invalid inputs, and hidden evaluation datasets.

Hard-coded paths, costs, traversal orders, or expected results will receive no credit for the affected component.

25. Important Notes

- Late submissions are not accepted unless an official extension has been granted.
- All required code must be written in Java.
- All required algorithms must be implemented by the group.
- Every group member must understand the complete project.
- The project must compile and run.
- Hard-coded algorithm results are not permitted.
- Hard-coding the complete required graph in Main.java is not permitted.
- External graph-algorithm libraries are not permitted.
- Google Maps APIs and route services are not permitted.
- Graphviz may be used only for visualization.
- A graph image alone does not prove algorithm correctness.
- All required validation and testing must be completed.
- External resources must be acknowledged in the report.
- Copied or unauthorized code may receive a grade of zero and may be reported under University academic-integrity policies.
- The group is responsible for explaining and defending every submitted component.

26. Timeline

Date	Required Progress
Tuesday, July 14	Read the complete statement. Create the Java project. Review the Graph ADT, adjacency lists, and adjacency matrices.
Wednesday, July 15	Implement Location, WeightedEdge, and Graph. Begin the adjacency-list representation.
Thursday, July 16	Complete the adjacency list. Implement the adjacency matrix and basic graph-operation tests.
Friday, July 17	First lab session. Demonstrate preliminary graph construction and CSV loading. Begin BFS and

Date	Required Progress
	predecessor-based path reconstruction.
Saturday, July 18	Complete BFS and BFS tests. Begin DFS and reachability.
Sunday, July 19	Complete DFS and cycle detection. Test connected, disconnected, cyclic, and acyclic graphs.
Monday, July 20	Implement Dijkstra using a priority queue. Test distance and travel-time optimization.
Tuesday, July 21	Complete Dijkstra testing, trace generation, path reconstruction, and negative-edge rejection.
Wednesday, July 22	Implement Bellman-Ford, early termination, negative-edge processing, and negative-cycle detection.
Thursday, July 23	Complete Graphviz export, path validation, algorithm comparison, report, screenshots, README, and final tests.
Friday, July 24	Complete the mandatory demonstration and submit the ZIP file and PDF report by 11:59 PM.

27. Minimum Required Demonstration Scenario

1. Load roads_positive.csv.
2. Display the adjacency list.
3. Display the adjacency matrix.
4. Run BFS from the University of Ottawa.
5. Find a minimum-edge path to Ottawa Airport.
6. Run DFS from the University of Ottawa.
7. Test an unreachable destination.
8. Detect a cycle.
9. Run Dijkstra using distance.
10. Validate the Dijkstra path.
11. Run Bellman-Ford on the same non-negative graph.
12. Confirm that the shortest-path costs match.
13. Load roads_negative.csv.
14. Show that Dijkstra rejects the graph.
15. Show that Bellman-Ford returns a valid result.
16. Load roads_negative_cycle.csv.
17. Show that Bellman-Ford detects the negative cycle.
18. Generate and display the required Graphviz files.

28. Expected Main Program Output

```
==================================================
CSI 2110 LAB 5 - MINIMAP ROUTE PLANNER
==================================================

Dataset loaded: roads_positive.csv
Vertices loaded: 15
Edges loaded: 28
Directed graph: Yes
Negative edge detected: No

Selected algorithm: DIJKSTRA
Source: University of Ottawa
Destination: Ottawa Airport
Metric: Distance

Shortest path:
University of Ottawa
-> Train Station
-> South Keys
-> Ottawa Airport
```

```
Total cost: 16.4
Number of edges: 3

Validation:
PASS - The path is structurally valid.
PASS - All edge directions are valid.
PASS - The reported cost is correct.

Graphviz output:
output/dijkstra_result.dot
output/dijkstra_result.svg
```

29. Final Submission Checklist

[ ] Graph ADT	[ ] Graphviz exporter
[ ] adjacency-list implementation	[ ] required DOT files
[ ] adjacency-matrix implementation	[ ] required SVG or PNG files
[ ] CSV loader	[ ] result.json
[ ] three required datasets	[ ] trace.csv
[ ] BFS implementation and minimum-edge path	[ ] systematic tests
[ ] DFS implementation	[ ] README.md
[ ] reachability operation	[ ] PDF report
[ ] cycle detection	[ ] names and student numbers
[ ] Dijkstra implementation	[ ] completed complexity table
[ ] Bellman-Ford implementation	[ ] answers to all written questions
[ ] negative-cycle detection	[ ] screenshots of execution and visualized graphs
[ ] path reconstruction	[ ] Dijkstra-Bellman-Ford comparison
[ ] path validator	[ ] complete project that compiles and runs